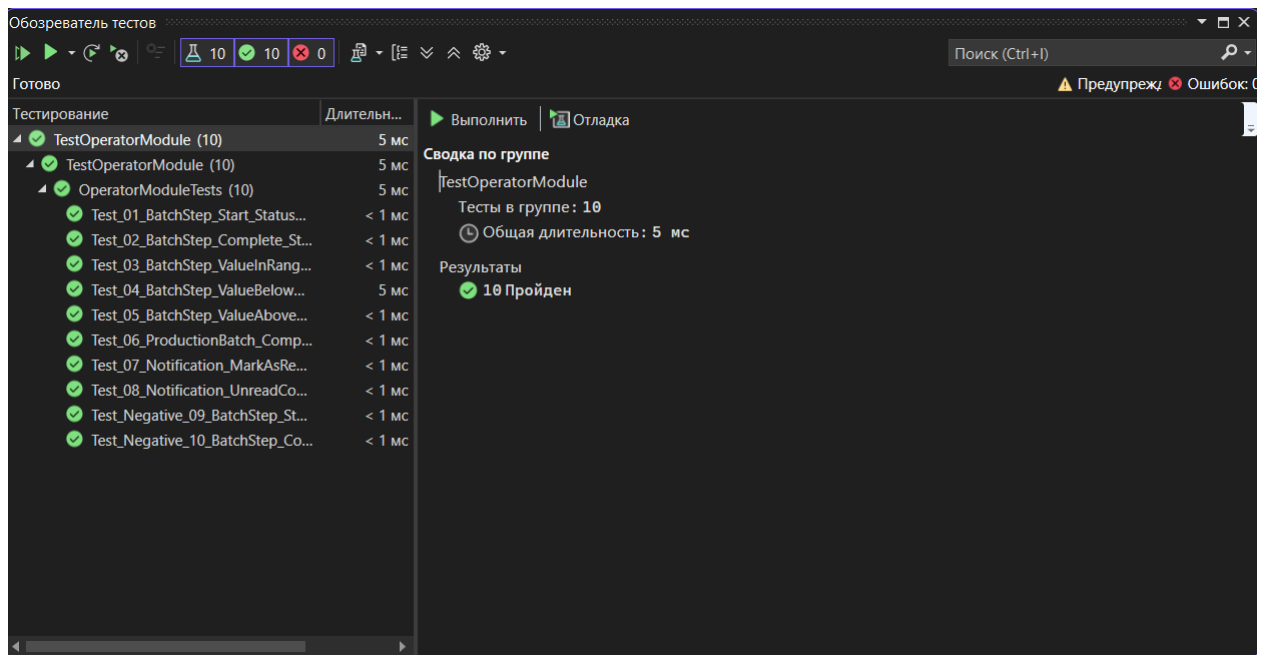




```
using Microsoft.VisualStudio.TestTools.UnitTesting;
```

```
using System.Collections.Generic;
```

```
namespace TestOperatorModule
```

```
{
```

```
    [TestClass]
```

```
    public class OperatorModuleTests
```

```
    {
```

```
        // ===== ТЕСТОВЫЕ МОДЕЛИ =====
```

```
        public class TestBatchStep
```

```
        {
```

```
            public long Id { get; set; }
```

```
            public string StepName { get; set; }
```

```
            public string Status { get; set; }
```

```
            public decimal? ActualValue { get; set; }
```

```
            public decimal? PlannedMin { get; set; }
```

```
            public decimal? PlannedMax { get; set; }
```

```
            public bool DeviationFlag { get; set; }
```

```
        }
```

```
        public class TestProductionBatch
```

```
{  
    public long Id { get; set; }  
    public string BatchNumber { get; set; }  
    public string Status { get; set; }  
}
```

```
public class TestNotification  
{  
    public long Id { get; set; }  
    public string Title { get; set; }  
    public bool IsRead { get; set; }  
}
```

```
// ===== ПОЗИТИВНЫЕ ТЕСТЫ (8 шт.) =====
```

```
[TestMethod]  
public void Test_01_BatchStep_Start_StatusChangesToInProgress()  
{  
    var step = new TestBatchStep { Status = "pending" };  
  
    step.Status = "in_progress";  
  
    Assert.AreEqual("in_progress", step.Status);  
}
```

```
[TestMethod]  
public void Test_02_BatchStep_Complete_StatusChangesToCompleted()  
{  
    var step = new TestBatchStep { Status = "in_progress" };  
  
    step.Status = "completed";
```

```
Assert.AreEqual("completed", step.Status);  
}
```

[TestMethod]

```
public void Test_03_BatchStep_ValueInRange_NoDeviation()
```

```
{  
    var step = new TestBatchStep  
    {  
        ActualValue = 82,  
        PlannedMin = 80,  
        PlannedMax = 85  
    };
```

```
    bool isDeviation = step.ActualValue < step.PlannedMin ||  
        step.ActualValue > step.PlannedMax;
```

```
    Assert.IsFalse(isDeviation, "Значение 82 в норме (80-85) → отклонения нет");  
}
```

[TestMethod]

```
public void Test_04_BatchStep_ValueBelowRange_DeviationDetected()
```

```
{  
    var step = new TestBatchStep  
    {  
        ActualValue = 75,  
        PlannedMin = 80,  
        PlannedMax = 85  
    };
```

```
    bool isDeviation = step.ActualValue < step.PlannedMin ||  
        step.ActualValue > step.PlannedMax;
```

```
Assert.IsTrue(isDeviation, "Значение 75 ниже нормы → отклонение");  
}
```

```
[TestMethod]
```

```
public void Test_05_BatchStep_ValueAboveRange_DeviationDetected()  
{  
    var step = new TestBatchStep  
    {  
        ActualValue = 95,  
        PlannedMin = 80,  
        PlannedMax = 85  
    };  
  
    bool isDeviation = step.ActualValue < step.PlannedMin ||  
        step.ActualValue > step.PlannedMax;  
  
    Assert.IsTrue(isDeviation, "Значение 95 выше нормы → отклонение");  
}
```

```
[TestMethod]
```

```
public void Test_06_ProductionBatch_CompleteAllSteps_StatusChanges()  
{  
    var batch = new TestProductionBatch { Status = "running" };  
  
    batch.Status = "quality_control";  
  
    Assert.AreEqual("quality_control", batch.Status);  
}
```

```
[TestMethod]
```

```
public void Test_07_Notification_MarkAsRead_IsReadTrue()  
{
```

```

var notification = new TestNotification { IsRead = false };

notification.IsRead = true;

Assert.IsTrue(notification.IsRead);
}

[TestMethod]
public void Test_08_Notification_UnreadCount_Calculation()
{
    var notifications = new List<TestNotification>
    {
        new TestNotification { IsRead = false },
        new TestNotification { IsRead = false },
        new TestNotification { IsRead = true }
    };

    int unreadCount = 0;
    foreach (var n in notifications)
    {
        if (!n.IsRead) unreadCount++;
    }

    Assert.AreEqual(2, unreadCount);
}

// ===== НЕГАТИВНЫЕ ТЕСТЫ (2 шт. - 20%) =====

[TestMethod]
public void Test_Negative_09_BatchStep_StartAlreadyStartedStep_ShouldFail()
{
    var step = new TestBatchStep { Status = "in_progress" };

```

```
bool canStart = step.Status == "pending";
```

```
Assert.IsFalse(canStart, "Шаг уже выполняется → нельзя начать повторно");
```

```
}
```

```
[TestMethod]
```

```
public void Test_Negative_10_BatchStep_CompleteNotStartedStep_ShouldFail()
```

```
{
```

```
var step = new TestBatchStep { Status = "pending" };
```

```
bool canComplete = step.Status == "in_progress";
```

```
Assert.IsFalse(canComplete, "Шаг не начат → нельзя завершить");
```

```
}
```

```
}
```

```
}
```